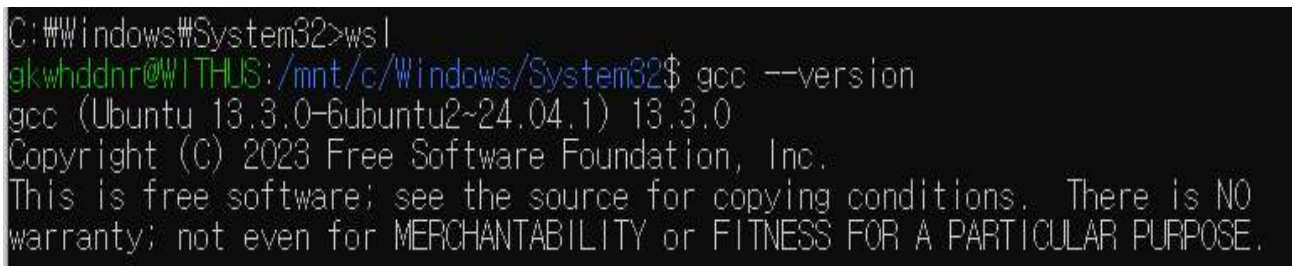
# 2026 Spring – GA3020: Homework 01

(공개일: 2026 년 3 월 15 일 (일) 18 시 / 제출기한: 2026 년 3 월 26 일 (목) 18 시 (KST))

숙제는 전자파일(PDF)로 LMS 시스템을 통해 지정된 기한까지 제출해야 합니다. 질문이 있다면 네이버톡/이메일로 시간을 정한 후 DB113 호로 오세요 (elec1020@kumoh.ac.kr).
문제의 답을 얻은 과정을 반드시 서술해야 합니다.

1. 리눅스 개발환경을 설정하자 (서술 필요 없음)
   - Mac OS 사용하는 경우: Terminal 을 열고 gcc 컴파일러가 동작하는지 확인한다. 없는 경우 설치하자 (https://joong-sunny.github.io/programming/gcc/).
   - Windows 사용하는 경우: WSL2 + UBUNTU 를 설치하자 (링크). gcc 컴파일러가 존재하는지 확인한 후, 경로를 설정해 놓자.

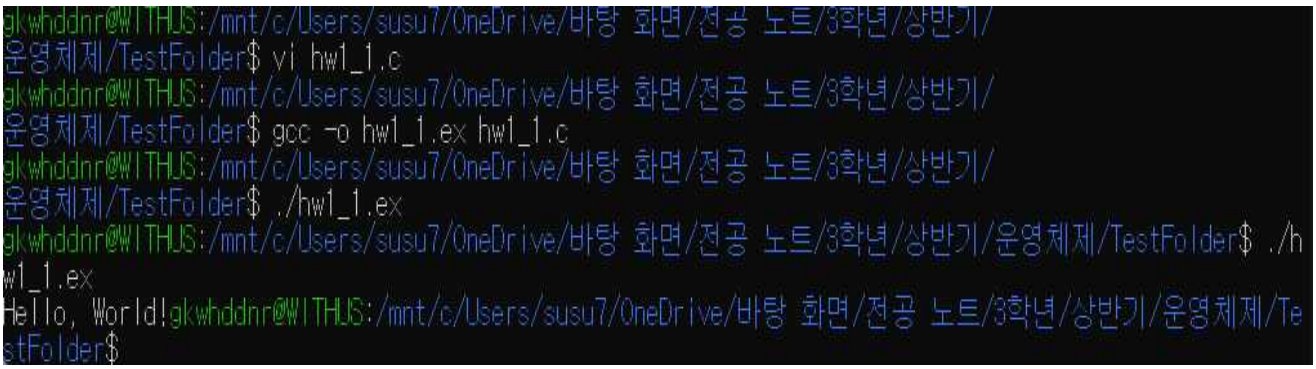
   (참고) Mac OS / Windows 모두 꼭 위의 방법을 따라야 하는 것은 아니다. Linux 에서 gcc 로 c 코드를 컴파일 할 수 있는 환경만 갖추면 됨.



2. 문제 1 에서 구축한 개발환경의 동작을 확인해 보자.

(1) 다음의 코드를 컴파일/실행 후 그 결과를 스크린샷으로 보이자 (서술 필요 없음).

```
/***************************************************** Source file: hw1_1.c
 Compile & Execution:
 $ gcc -o hw1_1.ex hw1_1.c
 $ ./hw1_1.ex
*****************************************************/ #include <stdio.h>

int main()
{
      printf("Hello, World!"); return 0;
}
```



(2) 다음의 코드를 컴파일/실행해 보자. 문제없이 잘 돌아가는 경우 스크린샷 등을 통해 결과를 보이고, 문제가 있다면 그 내용을 기술하자. 검색 등을 통해 문제를 해결할 수 있는 방법에 대해서도 알아보자.

```
/*****************************************************

 Source file: hw1_2.c
 Compile & Execution:
 $ gcc -o hw1_2.ex hw1_2.c
 $ ./hw1_2.ex
*****************************************************/
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>
#include <wait.h>

int main(void) {
        pid_t pid;
        int status, i;

        pid = fork();
        if (pid > 0)
        {
                sleep(1);
                return(0);
        }
        else if (pid == 0)
        {
                sleep(2);
                printf("(parent process) pid = %d", getppid());
                return -1;
        }
}
```

서술) 부모 프로세스(pid > 0)가 자식 프로세스 (pid == 0)보다 나중에 종료되어야 하는데 1초 후 부모 프러세스가 먼저 종료된다. 즉, 자식 프로세스가 모두 실행한 후 부모 프로세스가 마지막으로 종료되어야 하는데 부모 프로세스가 먼저 종료되므로 자식 프로세스는 '고아' 상태가 되면서 좀비 프로세스로 남겨진다. 따라서, 이를 해결하기 위해서 부모 프로세스에서 자식 프로세스보다 sleep()의 주기를 더 높게 설정하거나 부모 프로세스에 sleep() 함수 대신 wait() 함수로 대체하면 자식 프로세스가 먼저 종료되고 부모 프로세스가 종료된다.

```
gkwhddnr@WITHUS:/mnt/c/Users/susu7/OneDrive/바탕 화면/전공 노트/3학년/상반기/운영체제/TestFolder$ ./hw1_2.ex
(parent process) pid = 498gkwhddnr@WITHUS:/mnt/c/Users/susu7/OneDrive/바탕 화면/전공 노트/3학년/상반기/운영체제/TestFolder$
```

부모 프로세스 `sleep()` 함수를 2보다 높게 늘린 후 실행

```
gkwhddnr@WITHUS:/mnt/c/Users/susu7/OneDrive/바탕 화면/전공 노트/3학년/상반기/운영체제/TestFolder$ ./hw1_2.ex
(parent process) pid = 507gkwhddnr@WITHUS:/mnt/c/Users/susu7/OneDrive/바탕 화면/전공 노트/3학년/상반기/운영체제/TestFolder$
```

`sleep()` 대신 `wait(&status);` 함수로 대체 후 실행

```
gkwhddnr@WITHUS:/mnt/c/Users/susu7/OneDrive/바탕 화면/전공 노트/3학년/상반기/운영체제/TestFolder$ ./hw1_2.ex
(parent process) pid = 507gkwhddnr@WITHUS:/mnt/c/Users/susu7/OneDrive/바탕 화면/전공 노트/3학년/상반기/운영체제/TestFolder$ ps aux | egrep "Z|defunc
t"
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
gkwhddnr   510  0.0  0.0   3960  1920 pts/0    S+   15:46   0:00 grep -E --color=auto Z|defunct
```

프로세스 상태 확인

```
gkwhddnr@WITHUS: /mnt/c/Users/susu7/OneDrive/바탕 화면/전공 노트/3학년/상반기/운영
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdlib.h>
#include <wait.h>

int main(void) {
        pid_t pid;
        int status, i;

        pid = fork();
        if (pid > 0){
                sleep(1);
                // wait(&status);
                // sleep(3);
                return(0);
        }
        else if (pid == 0){
                sleep(2);
                printf("(parent process) pid = %d", getppid());
                return -1;
        }
```

`hw1_2.c` 코드

※ 참고자료

https://servermon.tistory.com/629 (좀비 프로세스 (Zombie Process)에 대해 알아보자.)

3. 다음은 프로그램 A, B, C 가 모두 도착해 있는 상태에서 배치 운영체제에 의해 처리되는 과정이다. 다음 2 개의 문항에 답하라. (아래 세 테이블은 연속된 시간 동안 A, B, C 의 작업 형태를 기술해 놓은 것임을 유의하자.)

| 시간 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| A 작업 | CPU | I/O | I/O | I/O | CPU | CPU | I/O | I/O | I/O | CPU |
| B 작업 | 대기 | 대기 | 대기 | 대기 | 대기 | 대기 | 대기 | 대기 | 대기 | 대기 |
| C 작업 | 대기 | 대기 | 대기 | 대기 | 대기 | 대기 | 대기 | 대기 | 대기 | 대기 |

| 시간 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
|---|---|---|---|---|---|---|---|---|---|---|
| A 작업 | CPU | | | | | | | | | |
| B 작업 | 대기 | CPU | CPU | I/O | I/O | I/O | I/O | CPU | I/O | I/O |
| C 작업 | 대기 | 대기 | 대기 | 대기 | 대기 | 대기 | 대기 | 대기 | 대기 | 대기 |

| 시간 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 |
|---|---|---|---|---|---|---|---|---|---|---|
| A 작업 | | | | | | | | | | |
| B 작업 | CPU | CPU | | | | | | | | |
| C 작업 | 대기 | 대기 | CPU | CPU | CPU | I/O | I/O | CPU | CPU | CPU |

(1) 프로그램 A, B, C 가 모두 메모리에 적재되어 있을 때 다중 프로그래밍 운영체제에서 3 개의 프로그램이 실행되는 과정을 그려보자. (프로그램 A, B, C 순서로 처리하되, 프로그램의 실행 중 입출력이 발생하면 다른 프로그램을 실행시킨다. 중단된 프로그램의 입출력 작업이 완료되었을 경우 현재 실행 중인 프로그램을 중단시키고, 입출력으로 중단된 프로그램을 다시 실행시킨다)

| 시간 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| A 작업 | CPU | I/O | I/O | I/O | CPU | CPU | I/O | I/O | I/O | CPU |
| B 작업 | 대기 | CPU | CPU | I/O | I/O | I/O | I/O | CPU | I/O | I/O |
| C 작업 | 대기 | 대기 | 대기 | CPU | 대기 | 대기 | CPU | 대기 | CPU | I/O |

| 시간 | 11 | 12 | 13 | 14 | 15 | 16 |
|---|---|---|---|---|---|---|
| A 작업 | CPU | | | | | |
| B 작업 | 대기 | CPU | CPU | | | |
| C 작업 | I/O | 대기 | 대기 | CPU | CPU | CPU |

(2) 실행 결과에 따라 다음 장에 있는 2 개의 표를 채워보자. 표에서 처리율은 단위 시간당 처리한 프로그램 개수를 의미하고, 응답 시간은 프로그램이 대기상태 에서 실행이 완료될 때까지 소요되는 시간을 의미하며, 평균 응답 시간은 3 개 프로그램의 응답 시간에 대한 평균값을 의미한다.

| | 배치 운영체제 | 다중 프로그래밍 운영체제 |
|---|---|---|
| 총 실행 시간 | 30 | 16 |
| CPU 유휴 시간 | ~~11~~ 14 | 0 |
| CPU 활용률 (utilization) | 16 ~~19~~/30 ≒ ~~63.3%~~ 53 | 16/16 = 100% |
| 처리율 (throughput) | 3 / 30 ≒ 0.1 | 3 / 16 ≒ 0.188 |

| | 배치 운영체제 | 다중 프로그래밍 운영체제 |
|---|---|---|
| 프로그램 A 의 응답 시간 | 11 | 11 |
| 프로그램 B 의 응답 시간 | 22 | 13 |
| 프로그램 C 의 응답 시간 | 30 | 16 |
| 프로그램의 평균 응답 시간 | ((11 +22 + 30)/3) 21 | ((11 + 13 + 16) / 3)13.33 |

4. 응용프로그램을 작성할 때 표준 라이브러리 함수(예 : printf())를 호출하는 것이 시스템 호출 함수(예 : write())를 호출하는 것보다 나은 방법인 이유를 설명하라.

서술)
printf() 함수를 사용하면 커널로 진입하는 횟수가 줄어들어 커널에 대한 부담이 적다. 응용프로그램의 입장에서 printf()의 빠른 return으로 인해 응답 속도가 빠르기 때문에 더 효율적이다.

※ write를 쓰는 경우는 매우 low-level 시스템 프로그래밍처럼 OS에 강하게 의존하는 레벨에서만 시스템 콜을 직접 쓰는 경우가 많음.

※ 참고자료

https://yummy0102.tistory.com/485 (시스템 콜(System Call))
https://didu-story.tistory.com/311 ([Operating System] (iOS) System Call (시스템콜, 시스템 호출이란?)))
https://velog.io/@lin419/printf%EC%9D%98-print-%EA%B8%B0%EB%8A%A5%EC%9D%84-write-%ED%95%A8%EC%88%98%EB%A1%9C-%EA%B5%AC%ED%98%84%ED%95%B4%EB%B3%B4%EC%9E%90 (printf의 print 기능을 write 함수로 구현해보자!)

5. 인터럽트 서비스 루틴이 커널 영역에 있어야 하는 이유는 무엇일까?

서술)
인터럽트 서비스 루틴(ISR)은 장치를 제어하거나 CPU의 인터럽트 플래그를 제어하는 등 특권 명령을 사용하기 때문에 커널 모드에서 실행해야 한다.

※ 참고자료

https://woovictory.github.io/2018/12/23/OS-The-Principle-Of-Interrupt/ ([운영체제] 인터럽트의 원리)
https://hongong.hanbit.co.kr/%EC%9A%B4%EC%98%81%EC%B2%B4%EC%A0%9C%EB%9E%80-%EC%BB%A4%EB%84%90%EC%9D%98-%EA%B0%9C%EB%85%90-%EC%9D%91%EC%9A%A9-%ED%94%84%EB%A1%9C%EA%B7%B8%EB%9E%A8-%EC%8B%A4%ED%96%89%EC%9D%84-%EC%9C%84%ED%95%9C/ (커널의 개념과 이중모드 & 시스템호출)

6. 컴퓨터 시스템과 운영체제의 중요한 목표 중 하나는 CPU 의 활용률을 극대화하는 데 있다. 인터럽트는 이 목적을 달성하게 하는데 중요한 역할을 한다. 인터럽트가 어떤 식으로 CPU 활용률을 높이는데 기여하는가?

서술)
인터럽트를 사용하면 입출력 장치가 입출력이 완료됐음을 알려줄 때까지 CPU는 다른 일을 할 수 있기 때문에 활용률은 높아진다.

※ 참고자료

https://mcrkgus.tistory.com/28 ([명품 운영체제] - Chapter02 연습문제 + 복합문제 풀이)

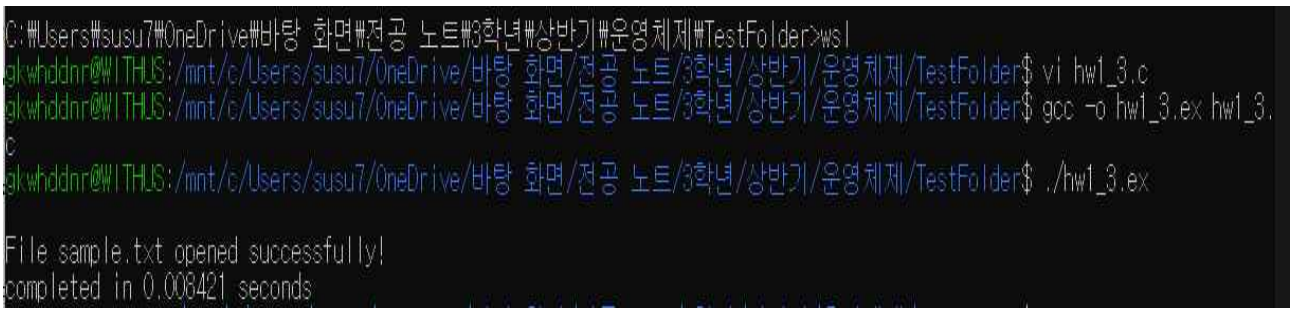
7. 아래의 C 코드는 System call 인 open, write, close 함수를 이용해 “Hello Educative User!”라는 문장을 sample.txt 파일에 1000 번 쓰는 작업을 한다. 코드를 자세히 살펴보고 다음 질문에 답해보자.

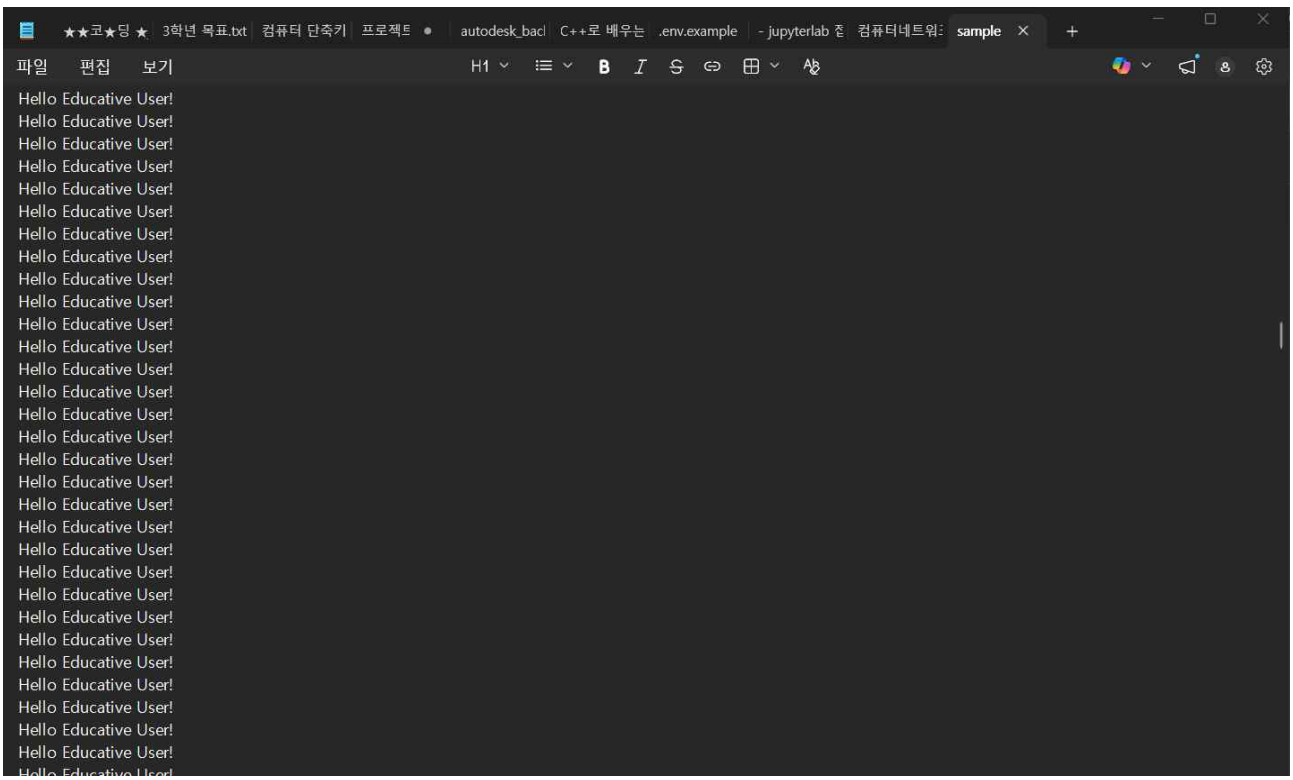
---

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <fcntl.h>
#include <string.h>
#include <time.h>

int main()
{
      char* fileName = "sample.txt"; int bytesWritten, i;
      int fd = open(fileName, O_RDWR); clock_t start, end;
      char *buffer = "Hello Educative User!\n";

      if(fd == -1)
      {

            printf("\nError Opening File!!\n"); exit(1);

      }
      else printf("\nFile %s opened successfully!\n", fileName);


      start = clock();

      for(i = 0; i < 1000; i++)
            bytesWritten = write(fd, buffer, strlen(buffer)); end = clock();

      printf("completed in %lf seconds\n", (double)(end-start)/CLOCKS_PER_SEC);

      return 0;
  }
```

---

(1) 위의 코드를 1 번 문제에서 구축한 개발환경에서 컴파일/실행했을 경우, 몇 초의 시간이 걸리는가? (컴파일 & 실행 후 나오는 메세지의 스크린샷을 첨부하시오) sample.txt 라는 이름의 빈 파일을 만든 다음 실행시켜야 함에 유의한다.



(2) C 언어의 파일 입출력 함수 (fopen, fwrite, fclose) 를 배웠을 것이다. 위의 코드를 system call 이 아닌 파일 입출력 함수로 바꿔 동일한 기능을 수행하도록 만든 다음 컴파일 & 실행시켜 보자. (1)에서 얻은 시간과 비교했을 때 무엇을 알 수 있는가? 그리고 그렇게 되는 이유는 무엇인가?

```
gkwhddnr@WITHUS:/mnt/c/Users/susu7/OneDrive/바탕 화면/전공 노트/3학년/상반기/운영체제/T
estFolder$ vi hw1_3.c
gkwhddnr@WITHUS:/mnt/c/Users/susu7/OneDrive/바탕 화면/전공 노트/3학년/상반기/운영체제/T
estFolder$ gcc -o hw1_3.ex hw1_3.c
gkwhddnr@WITHUS:/mnt/c/Users/susu7/OneDrive/바탕 화면/전공 노트/3학년/상반기/운영체제/T
estFolder$ ./hw1_3.ex

File sample.txt opened successfully!
completed in 0.000450 seconds
```

```
gkwhddnr@WITHUS: /mnt/c/Users/susu7/OneDrive/바탕 화면/전공 노트/3학년/상반기/운영체제/Tes...
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

int main()
{
        char* fileName = "sample.txt"; int i;
        FILE *fd = fopen(fileName, "r+"); clock_t start, end;
        char *buffer = "Hello Educative User!₩n";

        if(fd == NULL)
        {
                printf("₩nError Opening File!!₩n"); exit(1);
        }
        else printf("₩nFile %s opened successfully!₩n", fileName);

        start = clock();

        for(i = 0; i < 1000; i++)
                fwrite(buffer, sizeof(char), strlen(buffer), fd);

        fflush(fd);
        end = clock();

        fclose(fd);
        printf("completed in %lf seconds₩n", (double)(end-start)/CLOCKS_PER_SEC);

        return 0;
}
```

서술)

(1): 0.008421초,  (2): 0.000450초

1. 버퍼링: fwrite 함수는 사용자 공간의 stdio 버퍼에 데이터를 모았다가 버퍼가 차면 한 번에 큰 덩어리로 write 시스템 콜을 한 번 호출함.
2. 시스템 콜 오버헤드 감소: (1)의 경우 write를 1000번 시스템 콜을 수행하지만, stdio는 내부 버퍼 덕분에 적은 횟수의 시스템 콜로 같은 양의 데이터를 씀.

※ 참고자료

https://www.geeksforgeeks.org/c/fwrite-vs-write/ (fwrite()와 write()의 차이점)

https://jeseoyun.tistory.com/12 ([시스템프로그래밍/Linux] 2. File I/O(1))